

Metodologie di Programmazione (A-L)

Il semestre - a.a. 2025 – 2026

Strutture Dati I

a cura di Massimo La Morgia



SAPIENZA
UNIVERSITÀ DI ROMA

*Tutti i diritti relativi al presente materiale didattico ed al suo contenuto sono riservati a Sapienza e ai suoi autori (o docenti che lo hanno prodotto). È consentito l'uso personale dello stesso da parte dello studente a fini di studio. Ne è vietata nel modo più assoluto la diffusione, duplicazione, cessione, trasmissione, distribuzione a terzi o al pubblico pena le sanzioni applicabili per legge.

**I crediti sulle slide di questo corso sono riportati nell'ultima slide

Strutture Dati

parte I

A cosa serve una struttura dati?

- A memorizzare e organizzare i dati in memoria così da utilizzarli in modo efficiente



Una mano di carte



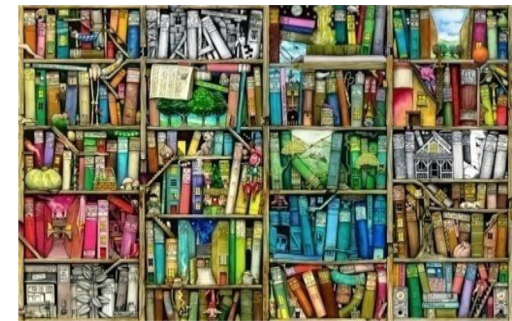
Una squadra di pallavolo



Uno spartito



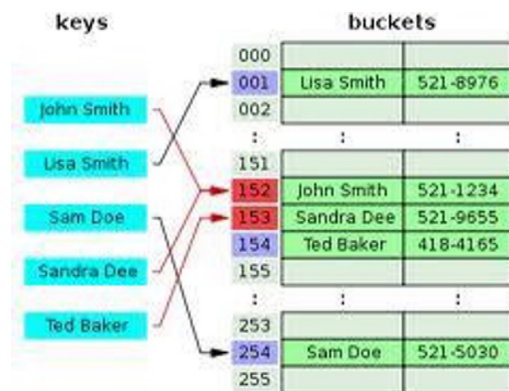
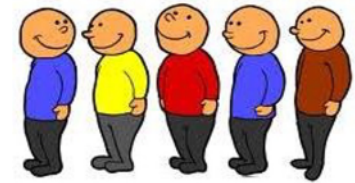
Una sequenza di canzoni



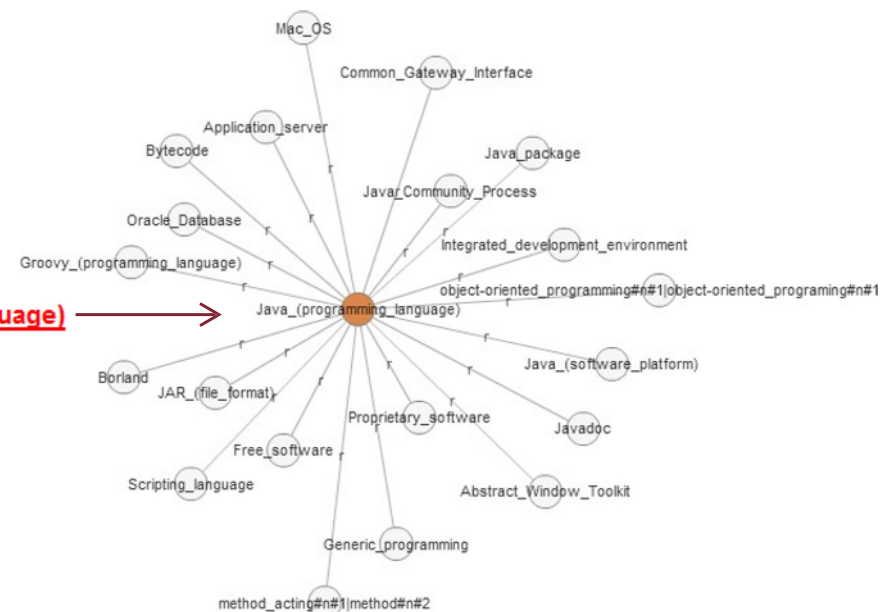
Una libreria

Caratteristiche di una struttura dati

- E' necessario mantenere un **ordine**?
- Gli oggetti nella struttura possono **ripetersi**?
- E' utile/necessario possedere una "**chiave**" per accedere a uno specifico oggetto?



Java (programming language) →

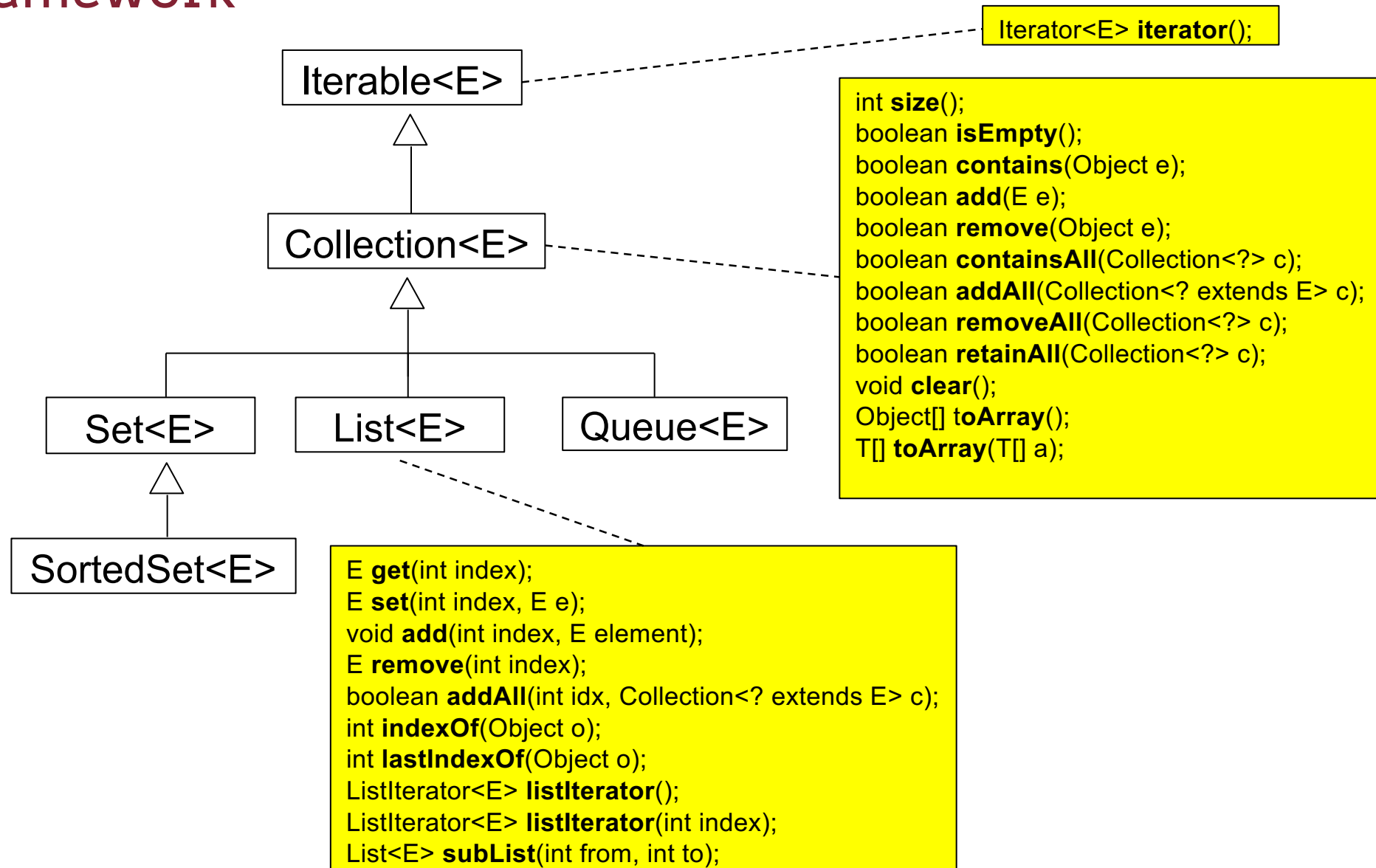


Le Collection

- Le **collezioni** in Java sono rese disponibili mediante il **framework delle collezioni** (Java Collection Framework)
- Strutture dati già pronte all'uso
 - con **interfacce** e **algoritmi** per manipolarle
- **Contengono** e **"strutturano" riferimenti** ad altri oggetti
 - Tipicamente tutti "dello stesso tipo"
- Alcune interfacce del framework:

Interfaccia	Descrizione
Collection	L'interfaccia alla radice della gerarchia di collezioni
Set	Una collezione senza duplicati
List	Una collezione ordinata che può contenere duplicati
Map	Associa coppie di (chiave, valore) , senza chiavi duplicate
Queue	Una collezione first-in, first-out che modella una coda

Gerarchia delle interfacce del Java Collection Framework



Iterazione *esterna* su una collezione

1. Mediante gli `Iterator`:

```
// "vecchio" stile, ma maggiore controllo
Iterator<Integer> i = collezione.iterator();
while(i.hasNext())
{
    int k = i.next();
    System.out.println(k);
}
```

Finché c'è ancora un elemento

Ottieni il prossimo elemento

2. Mediante il costrutto "`for each`":

```
// "nuovo" stile: for each, più elegante
for (Integer k : collezione)
    System.out.println(k);
```

Per ogni elemento k nella collezione

3. Mediante `indici` (solo su liste):

```
// su collezioni che implementano List: elemento per elemento
for (int j = 0; j < collezione.size(); j++)
{
    int k = collezione.get(j);
    System.out.println(k);
}
```

Accesso elemento per elemento

Iterazione *interna* su una collezione

- Mediante il metodo `Iterable.forEach` permette l'iterazione su qualsiasi collezione senza specificare come effettuare l'iterazione
- `forEach` prende in input un `Consumer<T>`, che è un'interfaccia funzionale con un solo metodo:

```
void accept(T t);
```

- Esempio:

```
List<Integer> l = List.of(4, 8, 15, 16, 23, 42);  
l.forEach(x -> System.out.println(x));
```

Modificare una lista durante l'iterazione

- Non è possibile utilizzare metodi di modifica durante un'iterazione:

```
for (int k : l)
    if (k == x) { l.remove(x); }
```

```
Exception in thread "main" java.util.ConcurrentModificationException
    at java.util.ArrayList$Itr.checkForComodification(Unknown Source)
    at java.util.ArrayList$Itr.next(Unknown Source)
    at ErroriComuni.main(ErroriComuni.java:27)
```

- ma è possibile utilizzare `Iterator.remove`:

```
Iterator<Integer> i = l.iterator();
while (i.hasNext())
    if (i.next() == x) i.remove();
```

Collezioni fondamentali



- Saranno molto utili nel tempo:
 - ArrayList
 - LinkedList
 - Estendono entrambe AbstractList
 - HashSet
 - TreeSet
 - Estendono entrambe AbstractSet
 - LinkedHashSet
 - Estende HashSet
 - HashMap
 - TreeMap
 - Estendono AbstractMap
 - LinkedHashMap
 - Estende HashMap

Le liste: ArrayList e LinkedList



- Basate su `List`, una sottointerfaccia di `Collection` e di `Iterable`
- Le classi liste sono: `ArrayList` e `LinkedList`
 - estendono `AbstractList` e implementano l'interfaccia `List`
- `ArrayList` implementa la lista mediante un array (eventualmente ridimensionato)
 - Capacità iniziale: 10 elementi
- `LinkedList` implementa la lista mediante elementi linkati

Le liste: ArrayList e LinkedList



0	1	2	3	4
23	3	17	9	42



Metodi di ArrayList



Method Summary

boolean	add (E e) Appends the specified element to the end of this list.
void	add (int index, E element) Inserts the specified element at the specified position in this list.
boolean	addAll (Collection <? extends E > c) Appends all of the elements in the specified collection to the end of this list, in the order that they are returned by the specified collection's Iterator.
boolean	addAll (int index, Collection <? extends E > c) Inserts all of the elements in the specified collection into this list, starting at the specified position.
void	clear () Removes all of the elements from this list.
Object	clone () Returns a shallow copy of this ArrayList instance.
boolean	contains (Object o) Returns true if this list contains the specified element.
void	ensureCapacity (int minCapacity) Increases the capacity of this ArrayList instance, if necessary, to ensure that it can hold at least the number of elements specified by the minimum capacity argument.
E	get (int index) Returns the element at the specified position in this list.
int	indexOf (Object o) Returns the index of the first occurrence of the specified element in this list, or -1 if this list does not contain the element.
boolean	isEmpty () Returns true if this list contains no elements.
int	lastIndexOf (Object o) Returns the index of the last occurrence of the specified element in this list, or -1 if this list does not contain the element.
E	remove (int index) Removes the element at the specified position in this list.
boolean	remove (Object o) Removes the first occurrence of the specified element from this list, if it is present.
protected void	removeRange (int fromIndex, int toIndex) Removes from this list all of the elements whose index is between fromIndex, inclusive, and toIndex, exclusive.
E	set (int index, E element) Replaces the element at the specified position in this list with the specified element.
int	size () Returns the number of elements in this list.
Object []	toArray () Returns an array containing all of the elements in this list in proper sequence (from first to last element).
<T> T[]	toArray (T[] a) Returns an array containing all of the elements in this list in proper sequence (from first to last element); the runtime type of the returned array is that of the specified array.
void	trimToSize () Trims the capacity of this ArrayList instance to be the list's current size.

Alcuni metodi aggiuntivi di LinkedList



Metodo	Descrizione
void addFirst (E e)	Aggiunge l'elemento in testa alla lista
void addLast (E e)	Aggiunge l'elemento in coda alla lista
Iterator<E> descendingIterator ()	Restituisce un iteratore che parte dall'ultimo elemento della lista e si sposta verso sinistra
E getFirst ()	Restituisce il primo elemento della lista
E getLast ()	Restituisce l'ultimo elemento della lista
E removeFirst ()	Rimuove e restituisce il primo elemento
E removeLast ()	Rimuove e restituisce l'ultimo elemento
E pop ()	Elimina e restituisce l'elemento in cima alla lista vista come pila
void push (E e)	Inserisce un elemento in cima alla lista vista come pila

Iterare sulle liste in entrambe le direzioni

- `listIterator()` restituisce un iteratore bidirezionale (interfaccia `ListIterator` che estende `Iterator`) per la lista

- Da sinistra verso destra:

```
ListIterator<Integer> i = l.listIterator();  
while(i.hasNext())  
    System.out.println(i.next());
```

- Da destra verso sinistra (se si specifica un intero, si parte da quella posizione: es.

```
list.listIterator(list.size()))
```

```
ListIterator<Integer> i = l.listIterator(l.size());  
while(i.hasPrevious())  
    System.out.println(i.previous());
```

ListIterator

Method Summary	
void	<u>add</u> (<u>E</u> e) Inserts the specified element into the list (optional operation).
boolean	<u>hasNext</u> () Returns <code>true</code> if this list iterator has more elements when traversing the list in the forward direction.
boolean	<u>hasPrevious</u> () Returns <code>true</code> if this list iterator has more elements when traversing the list in the reverse direction.
<u>E</u>	<u>next</u> () Returns the next element in the list.
int	<u>nextIndex</u> () Returns the index of the element that would be returned by a subsequent call to <code>next</code> .
<u>E</u>	<u>previous</u> () Returns the previous element in the list.
int	<u>previousIndex</u> () Returns the index of the element that would be returned by a subsequent call to <code>previous</code> .
void	<u>remove</u> () Removes from the list the last element that was returned by <code>next</code> or <code>previous</code> (optional operation).
void	<u>set</u> (<u>E</u> e) Replaces the last element returned by <code>next</code> or <code>previous</code> with the specified element (optional operation).

Insiemi: HashSet, TreeSet e LinkedHashSet

- Basate su `Set`, una sottointerfaccia di `Collection` e di `Iterable`
- Gli insiemi sono `Collection` che contengono elementi **tutti distinti**
- Le classi insiemi, `HashSet`, `TreeSet` e `LinkedHashSet`, estendono `AbstractSet` e implementano l'interfaccia `Set`
- `HashSet` memorizza gli elementi in una tabella di hash
- `TreeSet` memorizza gli elementi in un albero mantenendo un ordine sugli elementi
- `LinkedHashSet` memorizza gli elementi in ordine di inserimento

HashSet: Un esempio

- Salviamo nomi e cognomi in due insiemi:

```
HashSet<String> nomi = new HashSet<String>();  
HashSet<String> cognomi = new HashSet<String>();  
  
nomi.add("mario");  
cognomi.add("rossi");  
  
nomi.add("mario");  
cognomi.add("bianchi");  
  
nomi.add("mario");  
cognomi.add("verdi");  
  
nomi.add("luigi");  
cognomi.add("rossi");  
  
nomi.add("luigi");  
cognomi.add("bianchi");  
  
System.out.println(nomi);  
System.out.println(cognomi);
```

- Stampandoli, otteniamo:

```
[mario, luigi]  
[verdi, bianchi, rossi]
```


TreeSet: Un esempio

- Salviamo nomi e cognomi in due insiemi:

```
TreeSet<String> nomi = new TreeSet<String>();  
TreeSet<String> cognomi = new TreeSet<String>();  
  
nomi.add("mario");  
cognomi.add("rossi");  
  
nomi.add("mario");  
cognomi.add("bianchi");  
  
nomi.add("mario");  
cognomi.add("verdi");  
  
nomi.add("luigi");  
cognomi.add("rossi");  
  
nomi.add("luigi");  
cognomi.add("bianchi");  
  
System.out.println(nomi);  
System.out.println(cognomi);
```

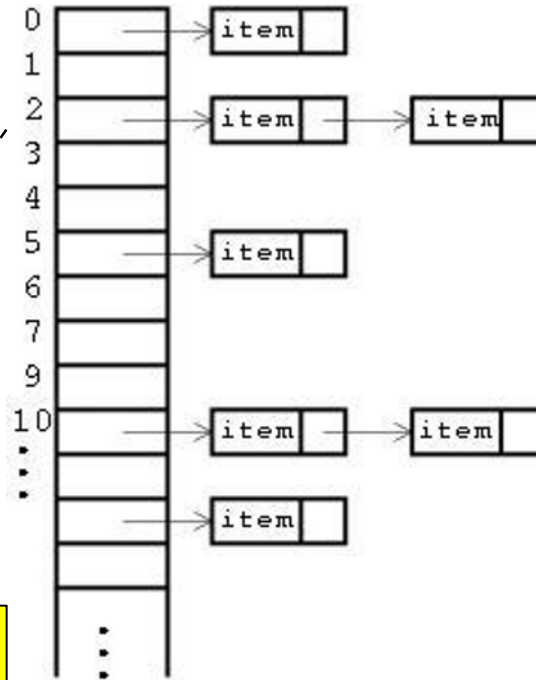
- Stampandoli, otteniamo:

```
[luigi, mario] -----  
[bianchi, rossi, verdi] -----
```

Ordinati secondo
l'ordinamento naturale
del tipo (String)

Come funziona un HashSet?

- Si fonda sul concetto di **tabella hash**



Questo numero è una
funzione
dell'**hashCode()**
dell'oggetto usato come
chiave



Esercizio

- Si definisca una classe **Canzone** i cui oggetti sono costruiti con nome e autore della canzone
- Si crei una classe **Raccolta** che contiene (ovvero i cui oggetti sono costruiti con) un insieme di canzoni
 - In una prima versione, si utilizzi HashSet per memorizzare l'insieme di canzoni
 - Si implementi correttamente **equals** e **hashCode** in modo da evitare duplicati di canzoni sostanzialmente uguali (ovvero con lo stesso nome e autore)
 - La classe è dotata di un metodo toString che restituisce la rappresentazione sotto forma di stringa dell'insieme di canzoni (analogamente, le canzoni sono dotato dello stesso metodo)
 - Si implementi quindi la classe **RaccoltaOrdinata** che utilizza TreeSet per mantenere un ordine sulle canzoni
 - Implementare un ordine sulle canzoni basato sul nome della canzone e, se questo è uguale, sul nome dell'autore

Mappe

- Una mappa mette in corrispondenza **chiavi** e **valori**
- **Non** può contenere **chiavi duplicate**
- `java.util.Map` è un'interfaccia implementata da `HashMap`, `TreeMap` e `LinkedHashMap`

L'interfaccia `java.util.Map`

Method Summary

Methods

Modifier and Type	Method and Description
void	<code>clear()</code> Removes all of the mappings from this map (optional operation).
boolean	<code>containsKey(Object key)</code> Returns <code>true</code> if this map contains a mapping for the specified key.
boolean	<code>containsValue(Object value)</code> Returns <code>true</code> if this map maps one or more keys to the specified value.
<code>Set<Map.Entry<K,V>></code>	<code>entrySet()</code> Returns a <code>Set</code> view of the mappings contained in this map.
boolean	<code>equals(Object o)</code> Compares the specified object with this map for equality.
<code>V</code>	<code>get(Object key)</code> Returns the value to which the specified key is mapped, or <code>null</code> if this map contains no mapping for the key.
int	<code>hashCode()</code> Returns the hash code value for this map.
boolean	<code>isEmpty()</code> Returns <code>true</code> if this map contains no key-value mappings.
<code>Set<K></code>	<code>keySet()</code> Returns a <code>Set</code> view of the keys contained in this map.
<code>V</code>	<code>put(K key, V value)</code> Associates the specified value with the specified key in this map (optional operation).
void	<code>putAll(Map<? extends K,? extends V> m)</code> Copies all of the mappings from the specified map to this map (optional operation).
<code>V</code>	<code>remove(Object key)</code> Removes the mapping for a key from this map if it is present (optional operation).
int	<code>size()</code> Returns the number of key-value mappings in this map.
<code>Collection<V></code>	<code>values()</code> Returns a <code>Collection</code> view of the values contained in this map.

HashMap: Un esempio

- Registriamo le frequenze di occorrenza delle parole in un testo:

```
public class MappaDelleFrequenze
{
    private Map<String, Integer> frequenze = new HashMap<String, Integer>();

    public MappaDelleFrequenze(File file) throws IOException
    {
        Scanner in = new Scanner(file);
        while(in.hasNext())
        {
            String parola = in.next();
            Integer freq = frequenze.get(parola);

            // parola mai incontrata (non presente nella mappa)
            if (freq == null)    frequenze.put(parola, 1);
            // incrementa il conteggio di frequenza della parola
            else                frequenze.put(parola, freq+1);
        }
    }
}
```

TreeMap: Un esempio

- Se vogliamo mantenere un **ordinamento sulle chiavi** (ovvero sulle parole) di cui calcoliamo la frequenza:

```
public class MappaDelleFrequenze
{
    private Map<String, Integer> frequenze = new TreeMap<String, Integer>();

    public MappaDelleFrequenze(File file) throws IOException
    {
        Scanner in = new Scanner(file);
        while(in.hasNext())
        {
            String parola = in.next();
            Integer freq = frequenze.get(parola);

            // parola mai incontrata (non presente nella mappa)
            if (freq == null)    frequenze.put(parola, 1);
            // incrementa il conteggio di frequenza della parola
            else                frequenze.put(parola, freq+1);
        }
    }
}
```

TreeMap
mantiene
l'ordinamento
delle chiavi

LinkedHashMap: Un esempio

- Se vogliamo mantenere un **ordinamento di iterazione** (ovvero sulle parole) di cui calcoliamo la frequenza:

```
public class MappaDelleFrequenze
{
    private Map<String, Integer> frequenze = new LinkedHashMap<String, Integer>();

    public MappaDelleFrequenze(File file) throws IOException
    {
        Scanner in = new Scanner(file);
        while(in.hasNext())
        {
            String parola = in.next();
            Integer freq = frequenze.get(parola);

            // parola mai incontrata (non presente nella mappa)
            if (freq == null)    frequenze.put(parola, 1);
            // incrementa il conteggio di frequenza della parola
            else                frequenze.put(parola, freq+1);
        }
    }
}
```

LinkedHashMap
mantiene
l'ordinamento di
iterazione sulle
chiavi

Confronto tra HashMap, TreeMap e LinkedHashMap

- Dato il seguente file:

a a e b c d e e b

- Se iteriamo su ciascuna delle tre strutture:

```
String s = "{";  
  
for (String chiave : frequenze.keySet())  
    s += chiave+"="+frequenze.get(chiave)+", ";  
  
s += "}";
```

otteniamo:

- **HashMap**: {d=1, e=3, b=2, c=1, a=2, }
- **TreeMap**: {a=2, b=2, c=1, d=1, e=3, }
- **LinkedHashMap**: {a=2, e=3, b=2, c=1, d=1, }

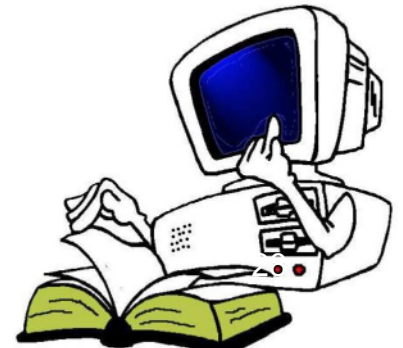
Senza
ordinamento
evidente

Ordinamento
naturale

Ordine di
inserimento

Chiavi e valori di una mappa

- E' possibile ottenere l'insieme delle chiavi di una mappa mediante il metodo `keySet`
- L'elenco dei valori mediante il metodo `values` (con ripetizione!)
- L'insieme delle coppie (chiave, valore) mediante il metodo `entrySet`
 - Restituisce un insieme di oggetti di tipo `Map.Entry<K, V>`
 - Per ogni oggetto (ovvero la coppia) è possibile conoscere la chiave (`getKey()`) e il valore (`getValue()`)



Un esempio di creazione di una TreeMap

- Il seguente codice crea una `TreeMap` e ne stampa l'insieme delle chiavi, la collezione dei valori e la collezione delle coppie (di tipo `Map.Entry<K, V>`):

```
Map<String, Integer> m = new TreeMap<String, Integer>();

m.put("a", 1);
m.put("b", 1);
m.put("c", 2);
m.put("d", 3);
m.put("e", 1);

System.out.println(m.keySet());
System.out.println(m.values());
System.out.println(m.entrySet());
```

- Output:

```
[a, b, c, d, e]
[1, 1, 2, 3, 1]
[a=1, b=1, c=2, d=3, e=1]
```

Alcuni metodi Java 8 e Java 9 nell'interfaccia `java.util.Map`

- `forEach(BiConsumer)`
 - Itera su ciascuna coppia (chiave, valore)
- `getOrDefault(chiave, valoreDefault)`
 - Restituisce il valore associato alla chiave o `valoreDefault` se la chiave non è presente
- `merge(chiave, valore, BiFunction)`
 - Se la chiave non contiene già un valore, imposta il valore specificato, altrimenti chiama una bifunzione che decide come mettere insieme il valore precedente con il valore passato in input
- `of(chiave, valore, chiave, valore, ..., chiave, valore)`
 - statico, crea una mappa immutabile dei tipi e con i valori corrispondenti



Esercizio: multimappa

- Creare una multimappa (ovvero una mappa che permette, a fronte della stessa chiave, di memorizzare più valori) esponendo i metodi:
- **get** e **put**
 - Si faccia uso dei metodi più avanzati disponibili in Java 8 (ad es. `getOrDefault` o `merge`)
- **keySet** e **values** (quest'ultimo restituisce l'elenco di tutti i valori presenti nella mappa, anche con ripetizione)
- una versione di **values** che restituisce una lista di elementi ordinata secondo una comparazione ad hoc
- **valueSet** che restituisce l'insieme di tutti i valori contenuti nella multimappa

`equals()` vs. `hashCode()`

- Entrambi sono metodi ereditati da `Object`
- `equals(Object o)` lo abbiamo già introdotto
- `hashCode()` invece richiede un approfondimento.
- Quando creiamo una classe, dobbiamo sovrascrivere sia `equals()` che `hashCode()`.
- Il contratto stabilisce che:
 - `a.equals(b)`  `a.hashCode()==b.hashCode()`
 - `a.hashCode()==b.hashCode()`  `a.equals(b)`

hashCode()

- quando invocato sullo stesso oggetto deve tornare sempre lo stesso valore.
- Se due oggetti sono uguali (per il metodo equals) allora anche il loro hashcode deve essere uguale.
- Se due oggetto non sono uguali, allora non devono produrre per forza hashcode diversi. Tuttavia, produrre valori diversi migliora le prestazioni.

equals() vs. hashCode() – la classe Persona

```
public class Persona {  
    String nome;  
    String cognome;  
    public Persona(String nome, String cognome){  
        this.nome=nome;  
        this.cognome=cognome;  
    }  
  
    ...  
  
}
```


equals() vs. hashCode() – la classe Persona

```
public class Persona {
    String nome;
    String cognome;
    public Persona(String nome, String cognome){
        this.nome=nome;
        this.cognome=cognome;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (obj == null) return false;
        if (getClass() != obj.getClass()) return false;
        Persona other = (Persona) obj;
        if (cognome == null) {
            if (other.cognome != null) return false;
        } else if (!cognome.equals(other.cognome)) return false;
        if (nome == null) {
            if (other.nome != null) return false;
        } else if (!nome.equals(other.nome)) return false;
        return true;
    }
}
```

equals() vs. hashCode() – la classe Persona

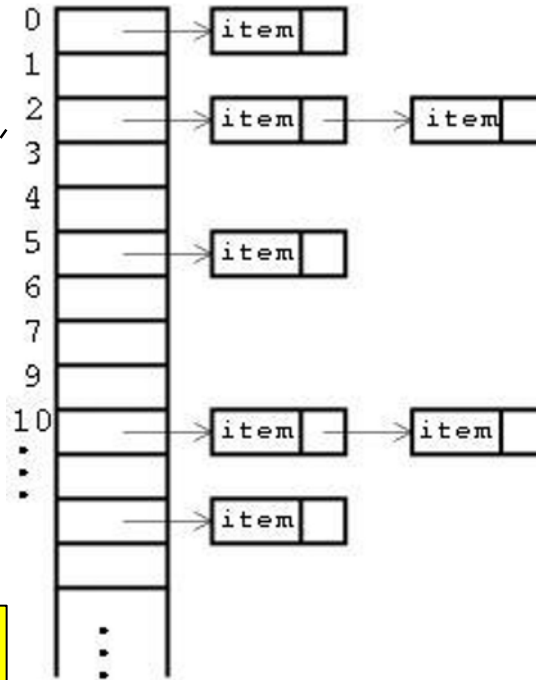
```
public class Persona {  
    String nome;  
    String cognome;  
    public Persona(String nome, String cognome){  
        this.nome=nome;  
        this.cognome=cognome;  
    }  
}
```

...

```
@Override  
public int hashCode() {  
    final int prime = 31;  
    int result = 1;  
    result = prime * result + ((cognome == null) ? 0 : cognome.hashCode());  
    result = prime * result + ((nome == null) ? 0 : nome.hashCode());  
    return result;  
}  
  
}
```

Come funziona un HashSet?

- Si fonda sul concetto di **tabella hash**



Questo numero è una
funzione
dell'**hashCode()**
dell'oggetto usato come
chiave

Credits

Le slide di questo corso sono basate sulle slide del Prof. Faralli che sono a loro volta una rielaborazione delle slide del Prof. Navigli

In aggiunta, le slide sono state revisionate dagli studenti borsisti della Facoltà di Ingegneria Informatica, Informatica e Statistica: Mario Marra e Paolo Straniero.

Font ad alta leggibilità Biancoenero® di biancoenero edizioni srl, disegnata da Umberto Mischi, con la consulenza di Alessandra Finzi, Daniele Zanoni e Luciano Perondi (Brevetto n. RM20110000128).

Disponibile gratuitamente per tutte le istituzioni e i privati che la utilizzino per scopi non commerciali.